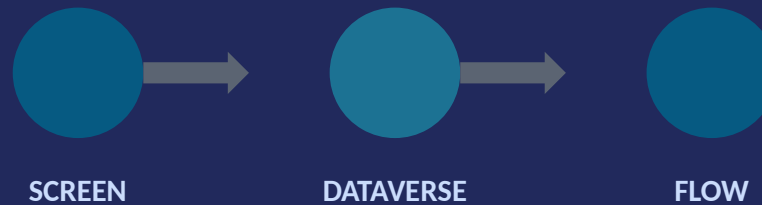


Power Platform Automation

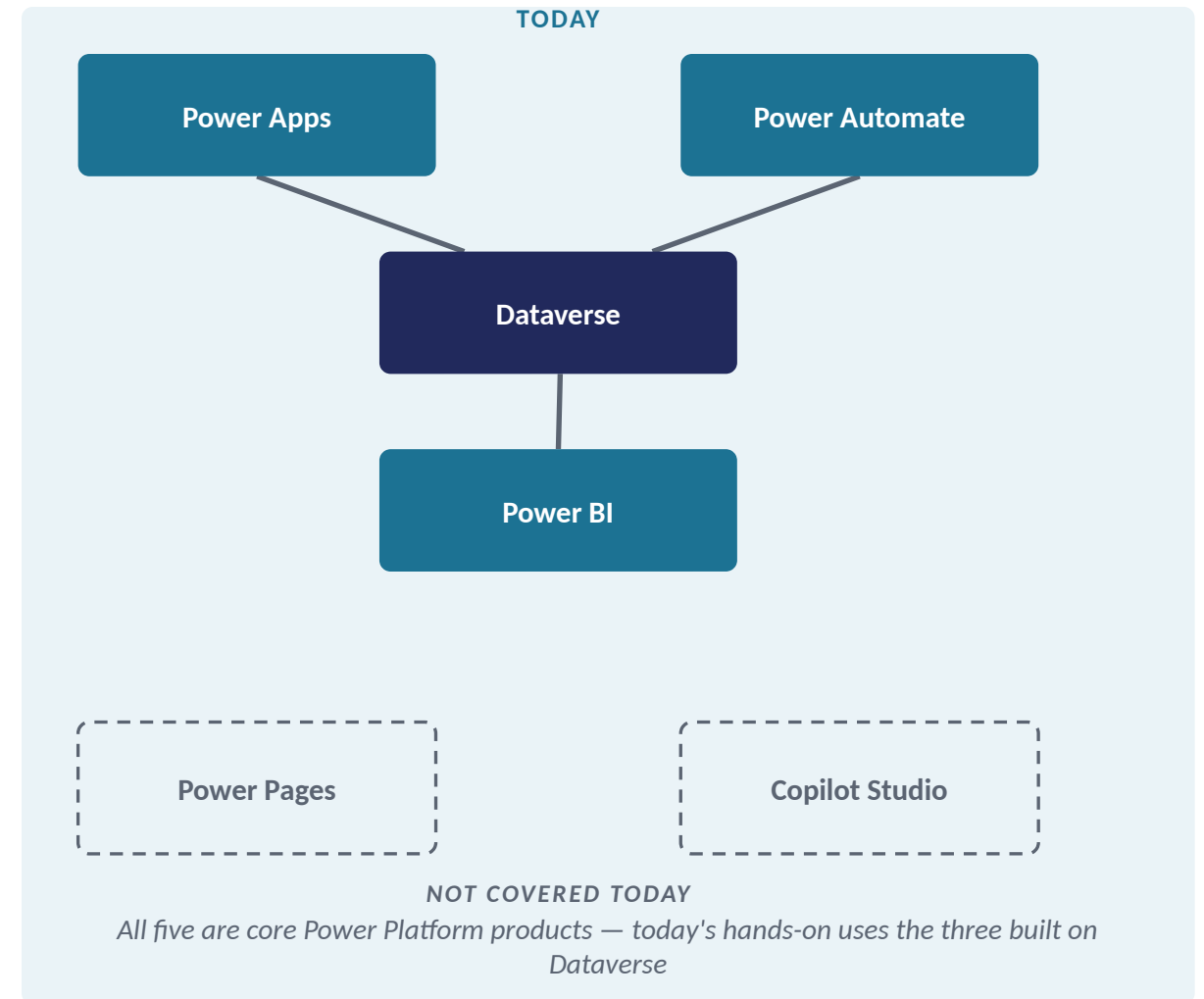
Canvas Apps & Dataverse • Power Automate Flows • Approvals & Power BI Overview



What Is the Power Platform?

OVERVIEW

- A family of low-code Microsoft tools, all built on one shared data layer called Dataverse — you configure instead of coding an entire program from scratch
- Power Apps: the screens and forms a person actually taps or clicks
- Power Automate: the automated logic that runs on its own, triggered by an event
- Power BI: turns that same stored data into reports and dashboards
- Power Pages (public websites) and Copilot Studio (AI chat agents) complete the family — both out of scope today



Power Apps & Dataverse Essentials

WHAT'S AHEAD

Canvas app design: screens and controls

Formulas: binding control behavior with Power Fx

Dataverse: tables, columns, and relationships

Building Every Internal Screen From Scratch Doesn't Fit the Task

PROBLEM

- Housekeeping and manager-review screens are small, internal, task-focused tools
- A full custom frontend means components, state, a build step, a hosting decision
- That engineering cost is disproportionate for a handful of internal staff
- The earlier full-stack work was the right call for a customer-facing product — not for this

custom frontend
components • state
build • deploy • host



3 housekeeping
staff

disproportionate engineering cost

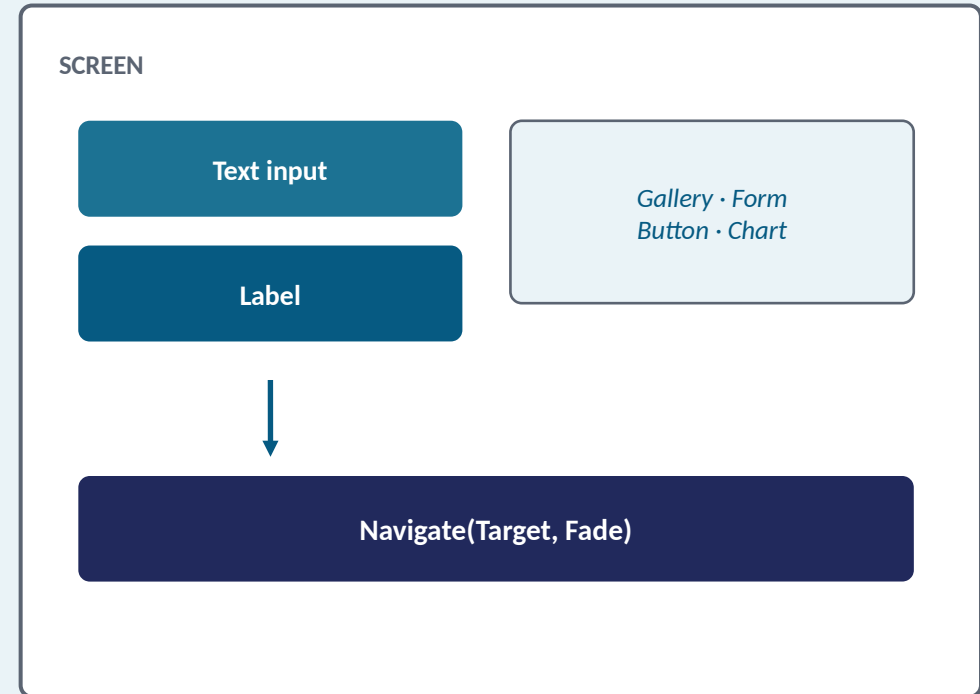
Right call for a customer-facing product — wrong size for this

Canvas App Design: Screens and Controls

SOLUTION

- A screen is a full-canvas container — the same idea as a page in a web app
- Controls are UI elements; the aspects you configure are properties
- Example controls: Button, Label, Gallery, Form, Chart
- `Navigate(Target, Fade)` moves between screens, wired to `OnSelect`

HANDS-ON Build one canvas app screen with a control bound to a formula.

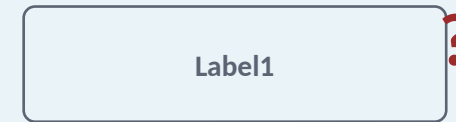


A control placed on a screen is still a static picture — nothing responds yet

A Static Screen Doesn't Respond to Anything Yet

PROBLEM

- A control placed on a screen shows nothing dynamic by itself
- The earlier frontend work made a control interactive with an event handler
- Canvas apps have no `onClick` function to write
- Something still has to make the label actually update and recolor itself



no event handler exists here

The earlier frontend's own fix for this doesn't exist in a canvas app

Formulas: Power Fx Binds Behavior Declaratively

SOLUTION

- Power Fx: "an open-source, low-code formula language based on Excel"
- A formula assigned to a property recalculates automatically, like a spreadsheet cell
- No leading "=" — the formula bar treats everything typed as a formula
- Example: Text = "Preference noted for: " & TextInput1.Text

SPREADSHEET



POWER APPS



recalculates the instant TextInput1 changes

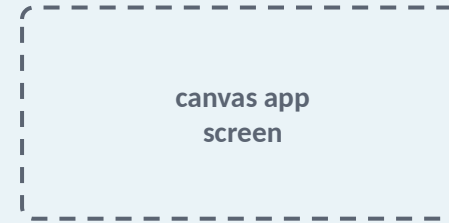
Same recalculation habit either way — the "cell" is now a control's property

HANDS-ON Build one canvas app screen with a control bound to a formula.

A Screen With Nowhere Durable to Keep Its Data

PROBLEM

- Nothing built so far persists data anywhere durable
- Data doesn't share across more than one user or survive closing the app
- The earlier relational schema solved this exact problem for the code-based stack
- A canvas app needs the same rigor, reachable from a low-code maker experience



no durable store



the earlier relational schema

A canvas app needs the same guarantee, through a low-code data layer

Dataverse: Tables and Columns for a Low-Code App

SOLUTION

- Dataverse: "securely store and manage data used by Power Apps business applications"
- Tables and columns, the same structural idea as a relational database
- Shared underneath Power Apps and Power Automate both
- Today's table: Guest Preference — a type, a detail, a flagged status

TABLE: GUEST PREFERENCE

preference type (choice)

detail (text)

flagged (yes/no)

Power Apps

Power Automate

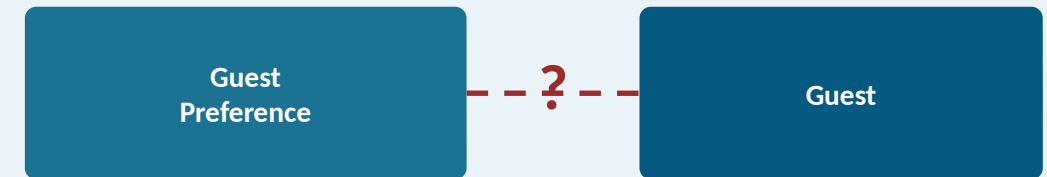
One shared table, read and written by both the screen and the flow ahead

HANDS-ON Model one Dataverse table with a relationship to a second table.

A Flat Table Can't Say Which Guest a Row Belongs To

PROBLEM

- Nothing stops two rows from referring to different, or no, guests
- This is the exact problem a foreign key solves in a relational schema
- The earlier Reservation-to-Guest relationship solved it once already
- Guest Preference, as modeled so far, is disconnected from Guest entirely



no connection between rows and their guest

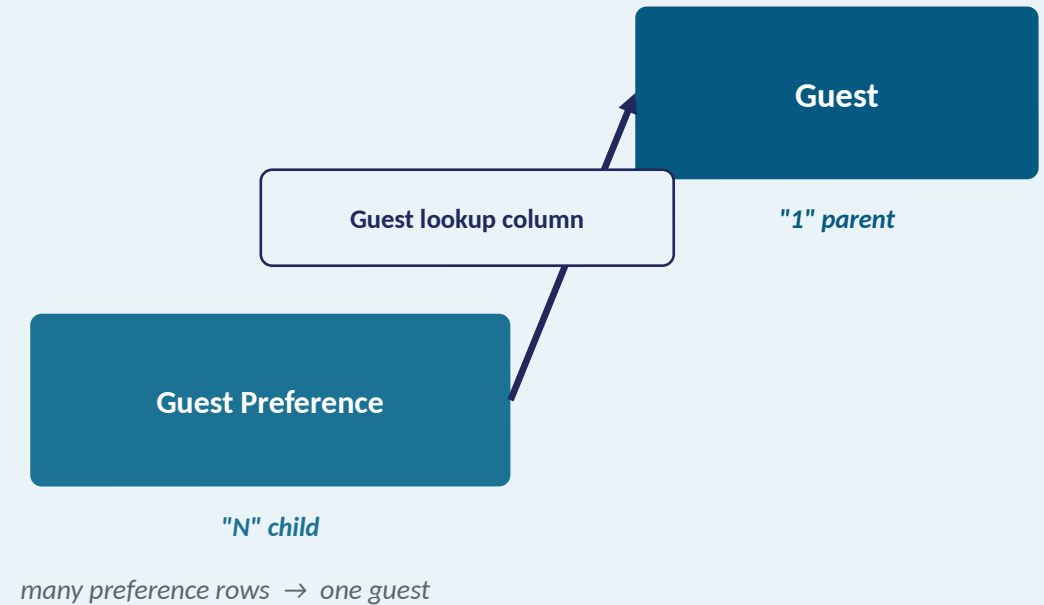
Solved once already in the relational schema — with a foreign key

Relationships: One-to-Many, Built With a Lookup Column

SOLUTION

- "Adding a lookup column ... creates a new 1:N relationship between the two tables"
- One-to-many: many child rows, one parent row each
- Guest Preference (child, "many") looks up Guest (parent, "one")
- Mirrors the Reservation-to-Guest relationship already designed once before

HANDS-ON Model one Dataverse table with a relationship to a second table.



Referential integrity, enforced by Dataverse instead of a SQL FOREIGN KEY

Power Automate Essentials

WHAT'S AHEAD

Flows, triggers, and actions

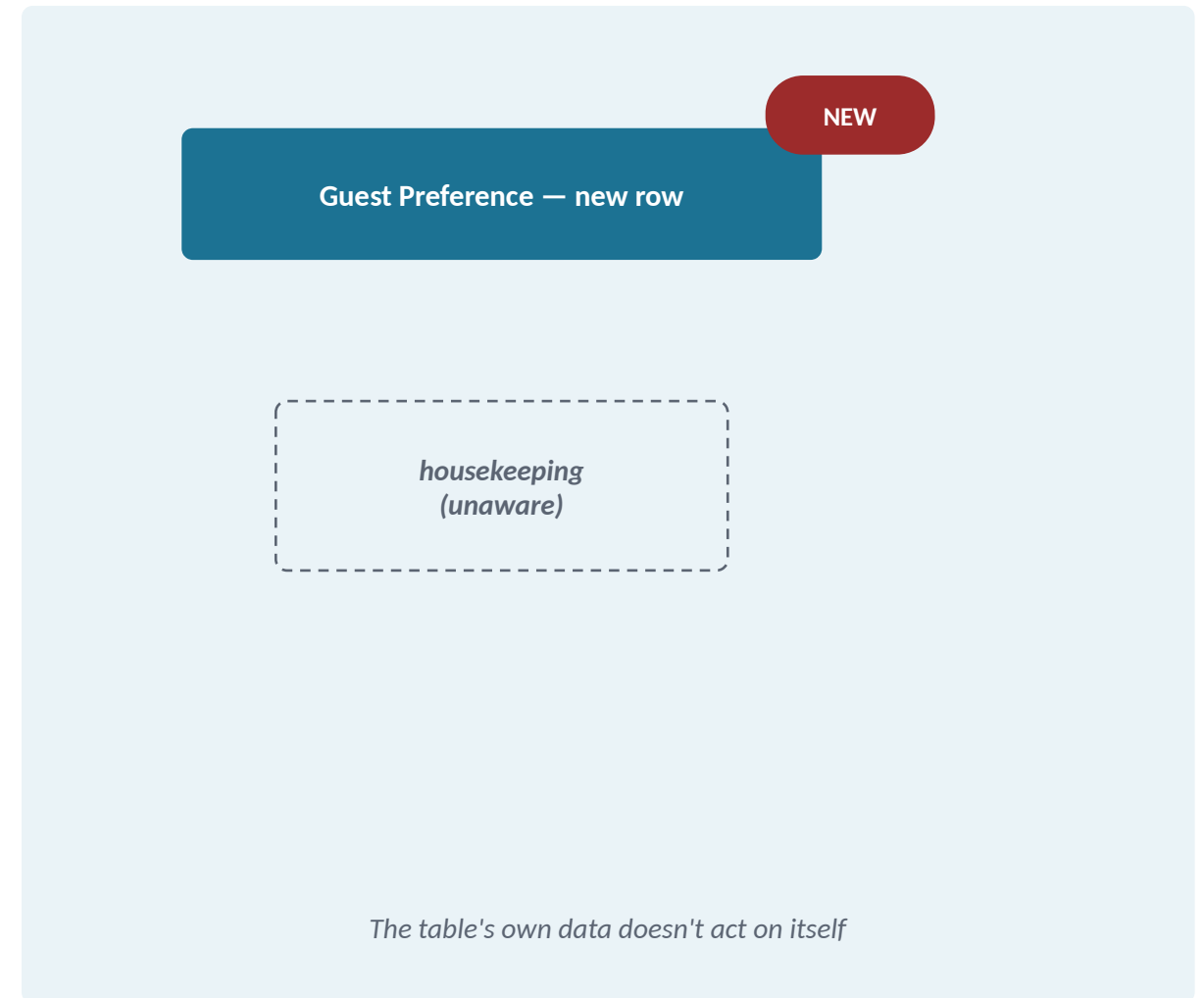
Approvals: a human decision inside an automatic flow

Power BI reporting, at overview level

A New Row Just Sits There, Unnoticed

PROBLEM

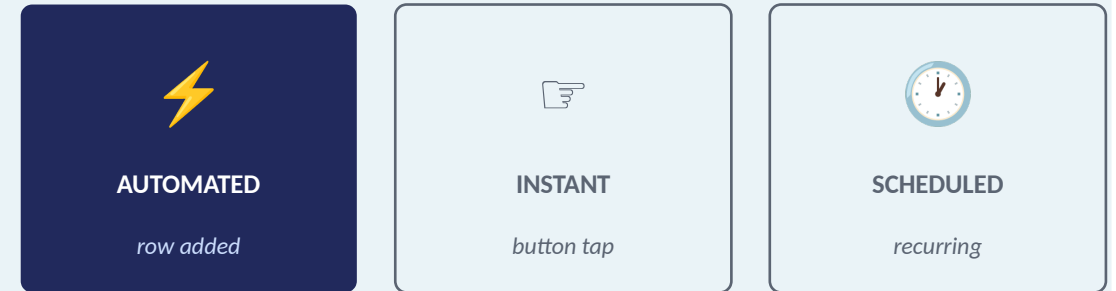
- A new flagged Guest Preference row appears in Dataverse
- Nobody on housekeeping sees it; nothing happens on its own
- The table is data, not automation
- Something has to notice the row appeared, and react



Flows and Triggers: What Starts an Automation

SOLUTION

- A trigger "is an event that starts a cloud flow"
- Automated: fires on an event (a Dataverse row added)
- Instant: fires when a person selects a button
- Scheduled: fires on a recurring schedule



TODAY'S CHOICE ↑

When a row is added — Guest Preference

The exact trigger that fires the moment a new preference row appears

HANDS-ON Build one flow with a trigger, an action, and an approval step, reading from the Dataverse table.

Detecting an Event Isn't the Same as Doing Anything About It

PROBLEM

- A trigger only notices that something happened
- The trigger's own Guest lookup is a record reference, not a readable name
- Nothing yet looks up who the guest actually is
- Nothing yet composes a message a person could read



an event, not yet a result

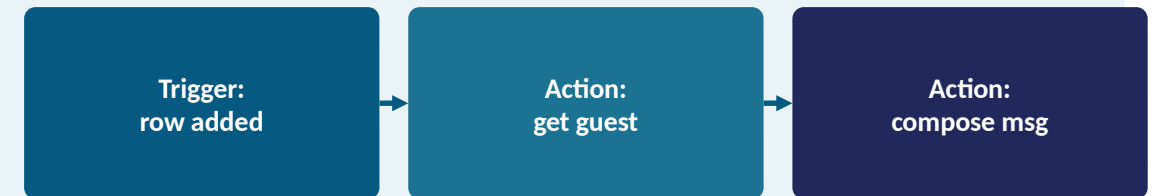
Something still has to look something up and compose a message

Actions: The Steps a Flow Actually Performs

SOLUTION

- "Actions are the events you want the flow to do after the trigger"
- Example: an email triggers a Teams notification action
- Get a row by ID on Guest, using the trigger's own lookup value
- Later actions can reference the trigger's and every prior action's output

HANDS-ON Build one flow with a trigger, an action, and an approval step, reading from the Dataverse table.



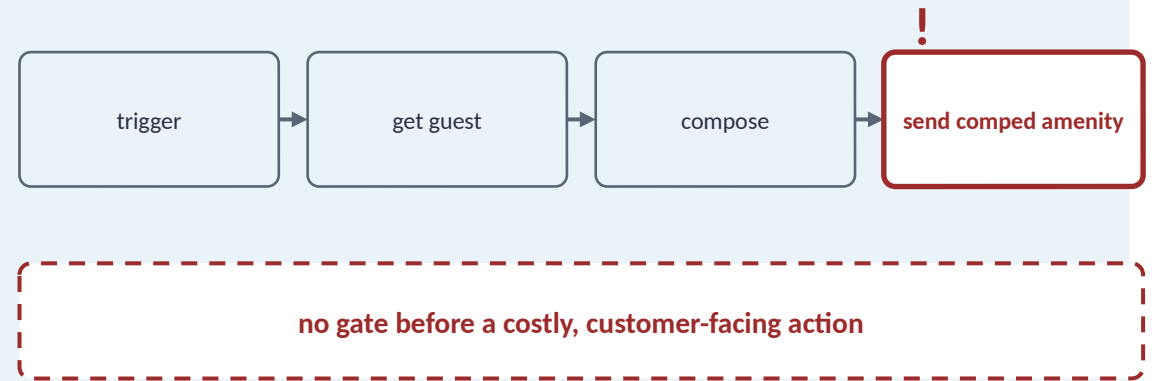
each action passes its output forward to the next

Module 18A's own Guest lookup column is what makes the middle box possible

Full Automation Has No Check Before a Costly Action

PROBLEM

- Everything so far runs to completion with nobody watching
- Right for a routine preference — a pillow type, a dietary note
- Wrong the moment the action has real cost or reaches a guest directly
- A comped, customer-facing amenity shouldn't go out unchecked



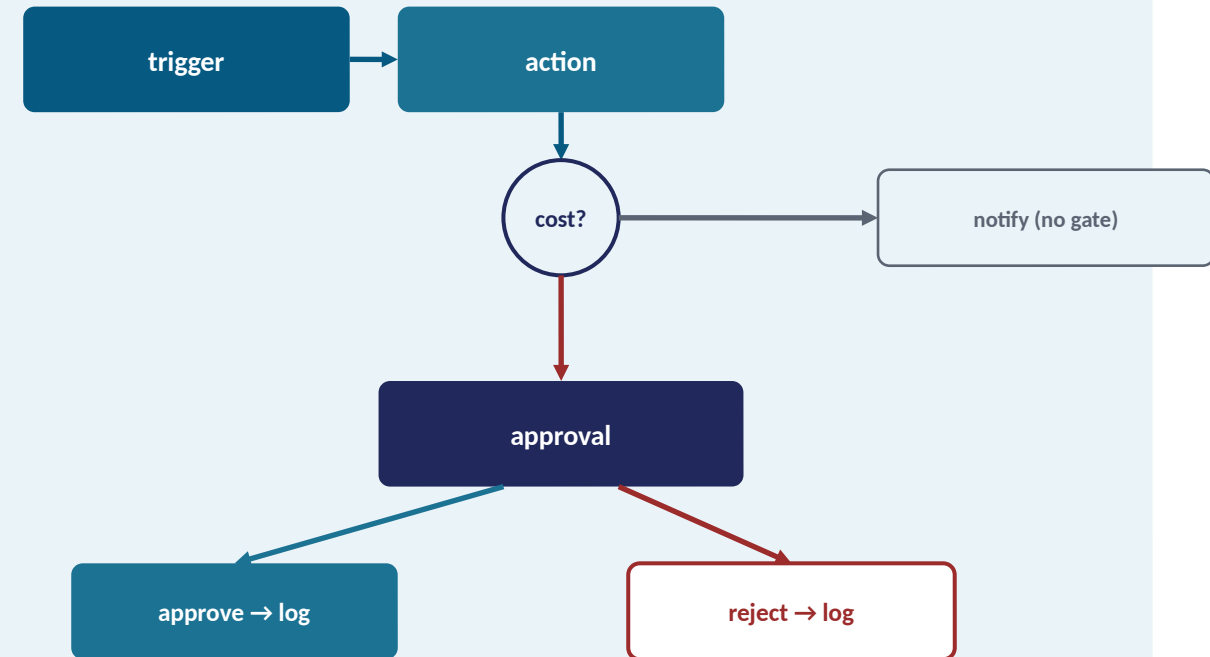
A generic teaching example — not any one team's real logic

Approvals: A Human Decision Inside the Flow

SOLUTION

- "Start and wait for an approval" — the flow pauses for a real response
- Five types: e.g. Approve/Reject (everyone, or first to respond), Sequential
- Approve routes one way, Reject routes another — neither is dropped
- A routine, no-cost preference skips this gate entirely

HANDS-ON Build one flow with a trigger, an action, and an approval step, reading from the Dataverse table.



Approve or reject — either way, the outcome is logged, never dropped

Applying the Flow to Your Team's Real Sprint 3 Task

APPLY

- This trigger-action-approval shape is a template, not the final answer
- A Power Automate team maps it onto its own real trigger and data
- Each team decides for itself whether its real flow needs a genuine approval
- A UiPath team treats today as a skills exercise for its own later build

HANDS-ON For a team choosing Power Automate as its Sprint 3 automated workflow, apply this flow to that capstone task.

TODAY'S GENERIC EXAMPLE

Guest Pref. trigger

get guest / compose

approval?

YOUR TEAM'S REAL FLOW

Aldwyn House — own trigger

Kitchens Collective — own trigger

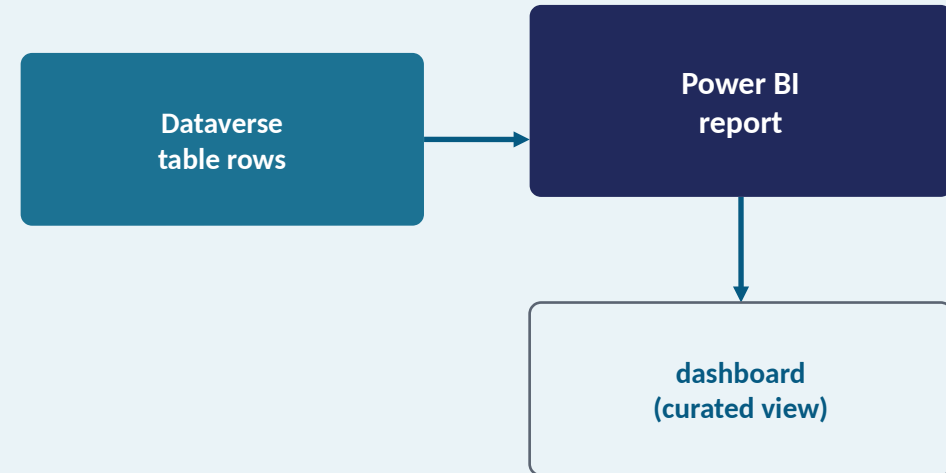
Meridian Stays — own trigger

same shape, real data — placeholder fields to fill in per team

Three teams complete their actual Sprint 3 deliverable here; two practice for the later UiPath build

Power BI: From One Row at a Time to a Pattern Across All of Them

- Every flow run leaves a trail of rows in Dataverse
- A report: "an interactive, multi-page document" built on a semantic model
- A dashboard is a single curated page pulled from one or more reports
- Dataverse connects to Power BI Desktop as a named data source



overview level only — no report built in class

Preparation for the individual assignment's own Power BI report, built outside class time